

# Phase IV: Software Testing

Project: Smart Parking Navigator

The testing phase of this project focused on checking the most important backend API functions of the Smart Parking Navigator application. The goal was to prepare clear test cases, write the related test code, execute the tests, and document the results in a way that shows whether the system behaves correctly.

## Task Overview

For this phase, the selected part of the software was the backend API component. This component was chosen because it controls important actions such as user registration, login, location management, parking search, nearby parking results, parking availability updates, and error handling.

The tests were prepared to check normal inputs, invalid inputs, protected routes, database behavior, and possible failure cases. The results were then verified using automated tests and live API requests.

## 1. Introduction to Testing

Software testing is the process of evaluating software to identify defects, bugs, or unexpected behavior. It helps developers check whether the application works as intended and whether the system gives correct results in different situations.

In the Smart Parking Navigator project, testing was important because the application depends on accurate parking information, location data, authentication, and parking availability. If one of these parts does not work correctly, users may receive wrong information or may not be able to use the system properly.

Testing also improves reliability and maintainability. When tests are available, future changes can be checked more easily because the tests show whether the main features are still working after updates are made.

## 2. Purpose of Testing

The purpose of testing in this project was to check that the API endpoints behave correctly under expected and unexpected conditions. Testing helped verify that each endpoint returns the correct response, stores data properly, protects private actions, and handles invalid requests in a controlled way.

The tests also helped confirm that important user actions work as expected. This includes registering an account, logging in, viewing the current user profile, creating locations, creating parking spaces, searching for parking, finding nearby parking, and updating parking availability.

Another purpose of testing was to make sure that errors are handled properly. For example, the system should reject duplicate registration, invalid login details, invalid coordinates, unknown parking locations, and requests that are made without authentication.

## 3. Focus on Testing a Single Component

The selected component for testing is the backend API component of the Smart Parking Navigator system. This component is responsible for receiving requests from users, processing them, communicating with the database, and returning the correct response.

This component is important because it supports the main features of the application. If the API does not work correctly, users may not be able to register, log in, search for parking spaces, view nearby parking, or receive accurate availability information.

The API component is also critical because it checks different conditions. It verifies whether a user is authenticated, whether the provided data is valid, whether a requested location or parking space exists, and whether parking availability values are acceptable.

Since the API directly affects the user experience and the correctness of the system, it was necessary to test it carefully.

#### 4. Preparing Test Cases

The test cases were designed to cover different possible API scenarios, including valid inputs, invalid inputs, protected actions, missing records, and boundary cases. Each test case clearly shows what is being tested, what input is used, and what result is expected.

| Test ID | Scenario                      | Input                                    | Expected Result                                      | Case Type |
|---------|-------------------------------|------------------------------------------|------------------------------------------------------|-----------|
| TC01    | Health check                  | GET /                                    | System returns status ok                             | Normal    |
| TC02    | Valid user registration       | Valid email and password                 | User is created successfully                         | Normal    |
| TC03    | Duplicate registration        | Already registered email                 | Conflict error is returned                           | Invalid   |
| TC04    | Valid login                   | Correct email and password               | Access token is returned                             | Normal    |
| TC05    | Wrong login password          | Correct email and wrong password         | Unauthorized error is returned                       | Invalid   |
| TC06    | Protected route without login | Request without token                    | Not authenticated response is returned               | Invalid   |
| TC07    | Create location               | Valid name, latitude, and longitude      | Location is created successfully                     | Normal    |
| TC08    | Invalid location coordinate   | Latitude outside allowed range           | Validation error is returned                         | Boundary  |
| TC09    | Create parking space          | Valid location id and spot values        | Parking space is created successfully                | Normal    |
| TC10    | Unknown location for parking  | Location id that does not exist          | Bad request is returned                              | Invalid   |
| TC11    | Filter available parking      | available_only set to true               | Only available parking spaces are returned           | Normal    |
| TC12    | Update parking availability   | available_spots set to zero              | Parking becomes unavailable                          | Normal    |
| TC13    | Invalid parking capacity      | available_spots greater than total_spots | Bad request is returned                              | Invalid   |
| TC14    | Search parking by name        | Search text for a location name          | Matching parking spaces are returned                 | Normal    |
| TC15    | Nearby parking search         | Latitude, longitude, and radius          | Nearby parking spaces are returned in distance order | Normal    |
| TC16    | Invalid nearby latitude       | Latitude greater than 90                 | Validation error is returned                         | Boundary  |
| TC17    | Missing parking record        | Parking id that does not exist           | Not found error is returned                          | Invalid   |

These test cases were selected because they cover the main ways a user or client application interacts with the system. They also cover cases where the system should reject incorrect data or prevent unauthorized actions.

#### 5. Writing Test Code

The test code was written using pytest and the FastAPI TestClient. Separate test functions were created for the different API behaviors. Helper functions were also used for repeated actions such as registering a user, logging in, creating a location, and creating a parking space.

A temporary SQLite database was used during testing. This allowed the tests to run safely without changing real project data. Each test started with a clean database state, which made the results easier to trust and repeat.

The following examples show parts of the implemented test code:

```
def test_health_check_returns_ok(client):
    response = client.get('/')
    assert response.status_code == 200
    assert response.json() == {'status': 'ok'}

def test_register_login_and_get_profile(client):
    registered = register_user(client)
    assert registered['email'] == 'driver@example.com'
    assert 'password' not in registered

    login_response = client.post(
        '/api/v1/auth/login',
        data={'username': 'driver@example.com', 'password': 'secret123'},
    )
    assert login_response.status_code == 200

def test_create_parking_rejects_unknown_location(client):
    headers = auth_headers(client)
    response = client.post(
        '/api/v1/parking/',
        json={'location_id': 999, 'name': 'Nowhere Garage', 'total_spots': 10, 'available_spots': 3},
        headers=headers,
    )
    assert response.status_code == 400
    assert response.json()['detail'] == 'Location not found'
```

Assertions were used to compare the actual API response with the expected result. For example, the tests checked response status codes, returned data, error messages, authentication behavior, and whether search log records were created.

## 6. Running Tests

The tests were executed by running the full test suite for the project. This included the API endpoint tests and the existing parking service tests. The command used for the test run was:

```
python -m pytest -q
```

The final test execution completed successfully with all 17 tests passing.

| Result Type | Meaning                                                                             | Example from This Project                                                                 |
|-------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Passed      | The system behaves correctly and produces the expected result.                      | A valid user registration returns a successful response and does not expose the password. |
| Failed      | The actual result does not match the expected result.                               | A test would fail if available parking filtering returned unavailable spaces.             |
| Error       | The system encounters a problem during execution and cannot complete the operation. | A database setup problem would stop the test before a normal API response is returned.    |

The API server was also started and checked with live HTTP requests. This confirmed that the server could accept real requests and respond correctly for the main workflow, including registration, login, creating locations, creating parking spaces, searching parking, finding nearby parking, updating availability, and handling invalid requests.

## 7. Test Coverage

Test coverage is important because it shows how much of the important system behavior has been checked. Good coverage does not only test successful cases. It also checks invalid input, protected routes, missing data, and boundary values.

The tests for this project covered the most important paths of the backend API component. They included normal cases, invalid cases, and edge cases that could affect the reliability of the application.

| Covered Area   | What Was Tested                                                                                                |
|----------------|----------------------------------------------------------------------------------------------------------------|
| Authentication | User registration, duplicate registration, login, invalid login, and current user profile access.              |
| Authorization  | Protected endpoints were checked to make sure unauthenticated users cannot create locations or parking spaces. |
| Locations      | Creating, listing, retrieving, and validating locations.                                                       |
| Parking Spaces | Creating, listing, retrieving, filtering, and updating parking spaces.                                         |
| Search         | Searching parking spaces by location name and recording search log information.                                |
| Nearby Parking | Finding parking spaces by coordinates and radius, including invalid coordinate handling.                       |
| Error Handling | Duplicate users, unknown records, invalid coordinates, unknown locations, and invalid capacity values.         |

Overall, the coverage is strong for the main API workflow and the most important failure scenarios. Future testing could still be improved by adding frontend interface tests, map display tests, and tests with larger sets of parking data.

The testing phase showed that the Smart Parking Navigator API works correctly for the main user actions. The automated tests passed successfully, and the live API verification confirmed that the server can start, accept requests, and return the expected responses.